

3.Spring AOP.

third in the series.

=====

Spring AOP (Aspect-Oriented Programming) - Detailed Explanation for Beginners

Spring AOP (Aspect-Oriented Programming) is a programming paradigm that allows separating cross-cutting concerns from business logic. It helps in modularizing concerns like logging, security, transaction management, and exception handling without cluttering the core business logic.

1. What is Aspect-Oriented Programming (AOP)?

AOP is a programming paradigm that enhances Object-Oriented Programming (OOP) by allowing cross-cutting concerns to be applied separately from the main application logic.

Example of Cross-Cutting Concerns

- **Logging:** Writing logs for method calls across the application.
- **Security:** Checking authentication before executing methods.
- **Transaction Management:** Handling database transactions automatically.
- **Performance Monitoring:** Measuring execution time of methods.

2. Spring AOP Terminologies

Spring AOP is built around the following key concepts:

Term	Description
------	-------------

Aspect	A modular part of a program that encapsulates a cross-cutting concern. This is main java class which defines things.
Advice	The actual code that runs when the aspect is triggered. (e.g., before, after, or around method execution).
Pointcut	A condition that specifies when the advice should be executed. refer point cut expression language explanation below. My take : Point cut is kind of expression to let spring know where should it scan for joinpoint .
Join Point	A specific point in the execution flow where advice can be applied (method execution, object initialization, etc.). method uses a join point to get info about the actual method being executed.
Weaving	The process of linking aspects with the application code.

Types of Advice in Spring AOP

Spring AOP provides different types of advice that control when the aspect executes.

Advice Type	Annotation	When It Runs
Before Advice	@Before	Runs before the method execution.
After Advice	@After	Runs after the method execution (regardless of success or failure).
After Returning Advice	@AfterReturning	Runs after the method completes successfully.
After Throwing Advice	@AfterThrowing	Runs when the method throws an exception.

Around Advice	@Around	Runs before and after the method execution (allows modifying behavior).
----------------------	---------	---

 Pointcut vs JoinPoint: Side-by-Side		
Concept	JoinPoint	Pointcut
What is it?	A specific execution point in the app	A condition to select which JoinPoints to use
Example	<code>ItemService.createItem()</code> method call	<code>execution(* com.example..*.create*(..))</code>
When it happens	At runtime, during method execution	At compile-time or load-time (based on expression)
Used in	Advice methods (as parameter)	In <code>@Before</code> , <code>@Around</code> , <code>@Pointcut</code> , etc.

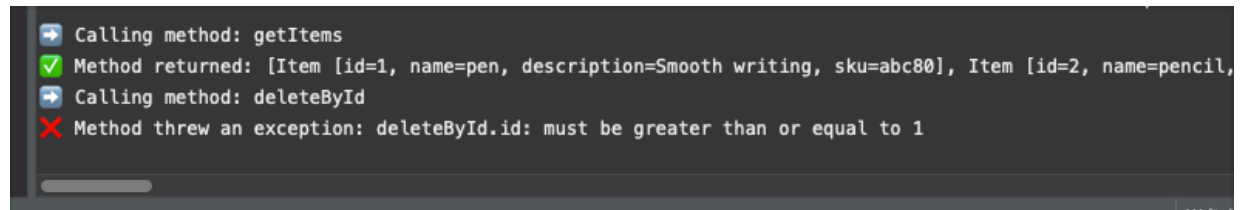
Feature	Join Point	Pointcut	
Definition	A specific point in the execution of the application (e.g., method call)	An expression that matches one or more join points	
What it does	Represents <i>where</i> advice can run	Specifies <i>which</i> join points to apply advice to	
Example	<code>execution of transferMoney()</code>	<code>execution(* com.example.service.*.*(..))</code>	
Who uses it	Used inside advice methods to get runtime context	Used to define advice targeting	
Spring AOP Scope	Only method executions are join points 	Pointcuts match method execution join points	

=====

Practical

I have created a new package Aspect and created LoggingAspect...

For logging aspect, I have created advices for controller method and I got following logs in eclipse



```
Calling method: getItem
Method returned: [Item [id=1, name=pen, description=Smooth writing, sku=abc80], Item [id=2, name=pencil,
Calling method: deleteById
Method threw an exception: deleteById.id: must be greater than or equal to 1
```

Did not do actual practical of @Around.

Example of @Around Advice

java

Copy

Edit

```
import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class ExecutionTimeAspect {

    @Around("execution(* com.example.demo.service.PaymentService.*(..))")
    public Object measureExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {
        long startTime = System.currentTimeMillis();

        Object result = joinPoint.proceed(); // Proceed with the actual method execution

        long endTime = System.currentTimeMillis();
        System.out.println(joinPoint.getSignature() + " executed in " + (endTime - startTime));

        return result;
    }
}
```

=====

What is *.*(..) in Pointcuts?

This is part of the **pointcut expression language** that tells Spring AOP *which methods* to match.

Let's break down this example:

execution(* com.example.service.*(..))

It looks cryptic at first, but it's really systematic:

Symbol/Pattern	Meaning
execution(...)	Designates a method execution join point

* (return type)	Match any return type
com.example.service	The package
* (class)	Match any class in that package
* (method)	Match any method name
(..)	Match any number and type of parameters

How Spring AOP Works Internally

Spring AOP uses **proxy-based implementation**:

- If the target object implements an interface, Spring AOP creates a **JDK dynamic proxy**.
- If the target object does not implement an interface, Spring AOP creates a **CGLIB proxy** (subclass-based proxy).

How proxies work in Spring AOP:

1. **Method Call → Proxy Object Intercepts the Call**
2. **Aspect Logic Executes (if applicable)**
3. **Actual Business Method Executes**
4. **Aspect Logic Executes After Completion**

=====

3. Spring AOP Implementations

Spring AOP can be implemented using:

1. **Proxy-based AOP (Spring Default Approach)**
2. **AspectJ (More powerful, compile-time weaving)**

Spring AOP uses runtime proxies for method interception, while AspectJ can

use compile-time or load-time weaving.

=====

1. Proxy-based AOP – *Spring's Default Approach*

What it is:

Spring AOP is **proxy-based**, meaning it creates **dynamic proxy objects** to wrap your beans. These proxies intercept method calls and apply any advice you've configured.

Key Concepts

- Uses **JDK Dynamic Proxies** (for interfaces) or **CGLIB proxies** (for classes).
- Applies AOP **at runtime**, using **reflection**.
- Only supports **method execution join points**.
- Works **only on Spring-managed beans**.

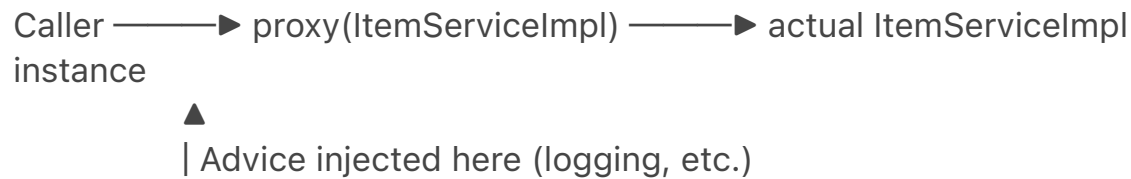
How it Works (Behind the Scenes)

Say you have a bean `ItemService` and an aspect that logs method calls:

```
public interface ItemService {  
    Item createItem(Item item);  
}
```

```
@Service  
public class ItemServiceImpl implements ItemService {  
    public Item createItem(Item item) {  
        // real logic  
    }  
}
```

Spring does this under the hood:



Example Output:

If you have @Before advice for createItem():

```
@Before("execution(* com.example.service.ItemService.createItem(..)")  
public void log() {  
    System.out.println("Calling createItem...");  
}
```

When itemService.createItem(...) is called:

- It hits the **proxy**
- Advice runs
- Then the real method executes



Limitations of Proxy-based AOP

Limitation	Explanation
✗ Only method execution join points	You can't intercept field access, constructors, etc.
✗ Only public methods	Private methods won't be proxied.
✗ Self-invocation doesn't trigger advice	Calling a method on the same class bypasses the proxy.
✗ Only Spring-managed beans	Won't work outside of Spring's container.



2. AspectJ – More Powerful AOP Framework



What it is:

AspectJ is a full-fledged AOP framework that **extends the Java language**

with new constructs. It supports **compile-time, post-compile (binary), or load-time weaving**.

Spring integrates with AspectJ to provide more powerful AOP capabilities.



Key Features of AspectJ

Feature	Description
✓ Intercepts more than just methods	Can weave into constructors, field access, initializers
✓ Works with private methods	Not limited to public/proxied methods
✓ Works outside Spring container	You don't need Spring context to apply AOP
✓ Compile-time or load-time weaving	Better performance, no proxy overhead
✓ Works with native Java classes	You can apply aspects to non-Spring beans or even 3rd-party classes



Types of Weaving in AspectJ

Weaving Type	Description
Compile-time	Aspects are woven into code when you compile.
Post-compile (Binary)	Weave aspects into existing .class files after compilation.
Load-time	Weave at class-loading time using a Java agent.



Example: Compile-Time AspectJ

AspectJ lets you write powerful aspects like:

```
aspect LoggingAspect {
    pointcut serviceMethods(): execution(* com.example.*Service.*(..));

    before(): serviceMethods() {
        System.out.println("Calling method: " + thisJoinPoint.getSignature());
    }
}
```

This can intercept:

- Private methods
- Static methods
- Field access (get/set)
- Object creation (new)

Proxy-based AOP vs AspectJ — Summary Table

Feature	Spring AOP (Proxy-based)	AspectJ
Weaving Type	Runtime (via proxy)	Compile-time / Load-time / Binary
Intercepts Method Calls	✓ Yes	✓ Yes
Intercepts Field Access	✗ No	✓ Yes
Works with Private Methods	✗ No	✓ Yes
Supports Constructor Interception	✗ No	✓ Yes
Self-invocation Supported	✗ No (bypasses proxy)	✓ Yes
Performance	Slower (reflection & proxies)	Faster (compiled bytecode)
External Tooling Required	✗ No	✓ Yes (for compile/load-time)
Spring Integration	✓ Excellent	✓ Excellent (with config)



When to Use What?

Use Case	Best Option
Simple cross-cutting concerns in Spring app (logging, security, etc.)	✓ Spring AOP
Need to intercept constructors, private methods, or 3rd-party libs	✓ AspectJ

Performance-critical applications with AOP	✓ AspectJ (compile-time weaving)
Minimal config, fast dev cycle	✓ Spring AOP

=====

Its working without adding any thing... I checked springboot starter web already has aop in dependency.

If you're using Spring Boot Starter (common case):

You likely already have this in your pom.xml:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

This dependency includes everything needed to:

- Use AOP annotations like @Aspect, @Before, @Around, etc.
- Register aspect classes as Spring beans.
- Enable AOP proxying under the hood.

⚠ If you're using other starters like spring-boot-starter-web, they **do not** include AOP support by default.

